Basics

Types, output, operators

Output & Input

`Console.WriteLine("Hi") ;`	print line
`Console.Write("Hi") ;`	print, no newline
`Console.ReadLine()`	read text
`$"Hi {name}"`	string interpolation

Variables & Types

`int x = 5;`	whole number
`double y = 3.14;`	decimal
`decimal m = 9.99m;`	money precise
`bool ok = true;`	true / false
`char c = 'a';`	one character
`string s = "text";`	text
`var v = 5;`	inferred type
`const int Max = 10;`	constant
`int? n = null;`	nullable

Operators

`+  -  *  /  %`	math
`==  !=  <  >`	compare
`&&  ||  !`	logic
`??  ?.`	null coalesce / safe
`int.Parse("5")`	text to int
`// one line  /* many */`	comments

Numbers & Math

Math, parsing, conversion

Math Class

`Math.Abs(x)  Math.Sqrt(x)`	absolute / root
`Math.Pow(a, b)`	power
`Math.Round(x, 2)`	round to 2 places
`Math.Floor(x)  Math.Ceiling(x)`	down / up
`Math.Max(a, b)  Math.Min(a, b)`	larger / smaller
`Math.Clamp(x, 0, 10)`	limit to a range
`Math.PI  Math.E`	constants

Parse & Convert

`int.TryParse(s, out var n)`	safe text to int
`double.Parse("3.14")`	text to double
`Convert.ToInt32("5")`	convert to int
`Convert.ToString(255, 2)`	number to binary text
`(int)3.9`	cast, cuts to 3
`(double)x`	cast to double

Limits & Random

`int.MaxValue  int.MinValue`	int limits
`long  ulong  byte`	other number types
`var r = new Random();`	random generator
`r.Next(1, 7)`	number 1 to 6
`Random.Shared.NextDouble()`	0.0 to 1.0
`x.ToString("F2")`	2 decimals as text

Strings & Collections

Text and lists

Strings

`s.Length`	length
`s.ToUpper()  s.ToLower()`	change case
`s.Trim()`	trim spaces
`s.Substring(1, 3)`	part of text
`s.Split(',')`	text to array
`s.Replace("a", "b")`	replace
`s.Contains("a")`	contains?
`string.Join(",", arr)`	array to text
`s.StartsWith("a")`	starts with?

List & Array

`int[] a = {1, 2, 3};`	array
`var l = new List<int>();`	list
`l.Add(4)  l.Remove(4)`	add / remove
`l.Count`	size
`l.Contains(2)  l.IndexOf(2)`	search
`l.Sort()  l.Reverse()`	order in place

Dictionary

`var d = new Dictionary<string,int>();`	create
`d["a"] = 1;`	set value
`d.TryGetValue("a", out var v)`	safe get
`d.ContainsKey("a")`	key exists?

Formatting

StringBuilder and format

String Formatting

`$"{x:F2}"`	2 decimals
`$"{x:N0}"`	thousands separator
`$"{x:C}"`	currency
`$"{x:P1}"`	percent
`$"{x,8}"`	right-align in 8
`$"{x,-8}"`	left-align in 8
`$"{n:D5}"`	pad with zeros

Compose Text

`string.Format("{0} {1}", a, b)`	classic format
`s.PadLeft(5, '0')`	pad left
`s.PadRight(10)`	pad right
`string.IsNullOrEmpty(s)`	null or empty?
`string.IsNullOrWhiteSpace(s)`	blank text?
`@"C:\path"`	verbatim string
`"""raw text"""`	raw string literal

StringBuilder

`var sb = new StringBuilder();`	fast text builder
`sb.Append("a");`	add text
`sb.AppendLine("a");`	add a line
`sb.Insert(0, "x");`	insert text
`sb.ToString()`	get the result

More Collections

Sets, queues, arrays

HashSet & Sorted

`var s = new HashSet<int>();`	unique items
`s.Add(1)  s.Contains(1)`	add / test
`s.UnionWith(t)  s.IntersectWith(t)`	set math
`var sl = new SortedList<int, string>();`	sorted pairs
`var sd = new SortedDictionary<K, V>();`	sorted dictionary

Queue & Stack

`var q = new Queue<int>();`	first in, first out
`q.Enqueue(1)  q.Dequeue()`	add / take
`q.Peek()`	look at next
`var st = new Stack<int>();`	last in, first out
`st.Push(1)  st.Pop()`	add / take
`st.Peek()`	look at top

Arrays & Spans

`int[,] g = new int[3, 3];`	2D array
`int[][] j = new int[3][];`	jagged array
`Array.Sort(a)  Array.Reverse(a)`	sort / reverse
`Array.IndexOf(a, 5)`	find index
`Array.Copy(a, b, a.Length)`	copy items
`a[1..3]  a[^1]`	range / last item
`Span<int> sp = a;`	fast slice view

Control Flow

Decide and repeat

Conditions

`if (x > 0) { } else { }`	branches
`x > 0 ? "yes" : "no"`	ternary
`switch (x) { case 1: break; }`	many cases
`x switch { 1 => "a", _ => "b" }`	switch expression
`x is > 0 and < 10`	pattern match

Loops

`for (int i = 0; i < 5; i++)`	classic loop
`foreach (var x in list)`	loop items
`while (cond) { }`	loop while true
`do { } while (cond);`	run at least once
`break;  continue;`	exit / skip

Errors

`try { } catch (Exception e) { }`	handle error
`finally { }`	always runs
`throw new Exception("msg");`	throw error
`using var f = File.OpenRead(p);`	auto dispose

Pattern Matching

Switch, records, is

Patterns

`x is int n`	type + variable
`x is not null`	not null check
`x is > 5 and < 10`	relational pattern
`x is 1 or 2 or 3`	one of values
`s is { Length: > 3 }`	property pattern
`p is (0, 0)`	positional pattern

Switch Expressions

`x switch { > 0 => "pos", _ => "neg" }`	expression
`case int n when n > 5:`	guard clause
`case null:`	match null
`default:`	no match
`var (a, b) = pair;`	deconstruct

Records & Init

`record P(string Name, int Age);`	immutable record
`p with { Age = 30 }`	copy and change
`p1 == p2`	value equality
`public int X { get; init; }`	set only at creation
`required string Name { get; set; }`	must be set

Methods & Classes

OOP basics

Methods

`static int Add(int a, int b) { }`	method
`int Add(int a, int b) => a + b;`	expression body
`void F(int a = 1)`	default value
`void F(ref int a)  out int b`	by reference
`void F(params int[] n)`	variable args

Classes

`class P { public string Name; }`	class
`public int Age { get; set; }`	auto property
`public P(string n) { Name = n; }`	constructor
`class B:  A { }`	inherit
`override  virtual  abstract`	polymorphism
`interface IA { void F(); }`	interface
`record R(int X, int Y);`	immutable record
`static  sealed  readonly`	modifiers

Namespaces

`using System;`	import
`namespace App;`	file namespace

Props & Interfaces

Generics and OOP

Properties

`public int X { get; private set; }`	private setter
`public int Y => x * 2;`	computed property
`private int _v;`	backing field
`public static int Count;`	shared field
`public const int Max = 10;`	constant

Interfaces & Abstract

`interface IA { void F(); }`	contract
`abstract class B { }`	cannot create
`abstract void F();`	must override
`class C : B, IA { }`	inherit + implement
`base.F();`	call parent method

Generics

`class Box<T> { }`	generic class
`T Max<T>(T a, T b)`	generic method
`where T : class`	reference type only
`where T : new()`	needs default constructor
`where T : IComparable<T>`	must be comparable
`default(T)`	default value of T

Delegates & Events

Lambdas and callbacks

Delegates

`delegate int Op(int a, int b);`	custom delegate
`Func<int, int> f = x => x * 2;`	returns a value
`Action<string> a = Console.WriteLine;`	no return value
`Predicate<int> p = x => x > 0;`	returns bool
`f(5)`	call it

Events

`event EventHandler Clicked;`	declare event
`Clicked?.Invoke(this, EventArgs.Empty);`	raise event
`btn.Clicked += OnClick;`	subscribe
`btn.Clicked -= OnClick;`	unsubscribe
`EventHandler<T>`	event with data

Lambdas & Local

`(a, b) => a + b`	expression lambda
`() => { DoWork(); }`	statement lambda
`static (x) => x * 2`	no captures
`int Local(int x) => x + 1;`	local function
`x => x?.Name`	null-safe access

LINQ & Async

Query and wait

LINQ

`using System.Linq;`	enable LINQ
`l.Where(x => x > 1)`	filter
`l.Select(x => x * 2)`	transform
`l.OrderBy(x => x)`	sort
`l.First()  l.FirstOrDefault()`	first item
`l.Any(x => x > 1)  l.All(...)`	any / all
`l.Sum()  l.Max()  l.Average()`	totals
`l.GroupBy(x => x.Key)`	group
`l.ToList()  l.ToArray()`	run query

Async

`async Task F() { }`	async method
`await Task.Delay(1000) ;`	wait 1 sec
`Task.WhenAll(t1, t2)`	wait for all
`async Task<int> F()`	return a value

Other

`var (a, b) = (1, 2);`	tuple deconstruct
`nameof(x)`	name as text
`is  as`	type check / cast

Files & JSON

Disk and serialization

File & Path

`File.ReadAllText(p)`	read whole file
`File.WriteAllText(p, s)`	write whole file
`File.ReadAllLines(p)`	lines to array
`File.AppendAllText(p, s)`	add to file
`File.Exists(p)  File.Delete(p)`	check / delete
`Path.Combine(a, b)`	build a path
`Path.GetExtension(p)`	file extension

Directory

`Directory.GetFiles(d)`	list files
`Directory.CreateDirectory(d)`	make folder
`Directory.Exists(d)`	folder exists?
`Environment.CurrentDirectory`	working folder

JSON & Streams

`JsonSerializer.Serialize(obj)`	object to JSON
`JsonSerializer.Deserialize<T>(s)`	JSON to object
`using var r = new StreamReader(p);`	read as a stream
`r.ReadLine()`	read one line
`await File.ReadAllTextAsync(p)`	async read

Dates & Time

DateTime and TimeSpan

DateTime

`DateTime.Now`	local time
`DateTime.UtcNow`	UTC time
`DateTime.Today`	today at midnight
`new DateTime(2026, 5, 20)`	make a date
`d.Year  d.Month  d.Day`	parts
`d.DayOfWeek`	weekday

Math with Dates

`d.AddDays(7)`	add days
`d.AddMonths(-1)`	go back a month
`(d2 - d1).TotalDays`	days between
`DateTime.Compare(a, b)`	compare dates

Text & Spans

`d.ToString("yyyy-MM-dd")`	date to text
`DateTime.Parse(s)`	text to date
`DateTime.TryParse(s, out var d)`	safe parse
`TimeSpan.FromMinutes(90)`	a time gap
`ts.TotalSeconds`	gap in seconds
`DateOnly.FromDateTime(d)`	date without time

Modern C#

Newer language features

Concise Code

`var x = new();`	target-typed new
`string? s = null;`	nullable reference
`x ??= 5;`	assign if null
`s?.Length ?? 0`	safe + default
`using var f = File.OpenRead(p);`	dispose at scope end
`global using System;`	project-wide import

Collections & Ranges

`int[] a = [1, 2, 3];`	collection expression
`[..a, 4]`	spread items
`a[^1]  a[..2]`	from end / slice
`foreach (var (k, v) in dict)`	deconstruct pairs

Classes

`class P(string name) { }`	primary constructor
`file class Helper { }`	visible in this file
`ref struct S { }`	stack-only type
`readonly struct V { }`	immutable struct
`init`	set only in initializer

Tips & dotnet

CLI and habits

dotnet CLI

`dotnet new console`	new project
`dotnet run`	build and run
`dotnet build`	compile
`dotnet add package X`	add package
`dotnet test`	run tests
`dotnet publish -c Release`	release build

Good Habits

`Enable nullable reference types`	fewer null bugs
`Use async all the way`	no .Result
`Use var when type is obvious`	shorter code

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily